

东南大学考试卷(A卷)

课程名称 数据库原理 考试学期 07-08-2 得分
适用专业 计算机科学与技术 考试形式 开卷 考试时间长度 120 分钟

(可携带教材、授课 PPT 讲义、笔记)

1. 在 DBMS 中,通常采用多级数据模式,例如概念模式、外模式和内模式,简述数据库系统中的多级数据模式对数据独立性的影响。(8%)

1. 简述数据库系统中的多级数据模式对数据独立性的影响。

1. 概念模式: 概念模式是用逻辑数据模型对数据库的一个单位的数据的描述。概念模式的设计是数据库设计的最基本任务。概念模式也称为逻辑模式。

2. 外模式: 外模式是用逻辑数据模型对用户所用到的那部分数据的描述。每个用户所感知的数据不完全一样。另外,从保密的观点出发也不宜让用户接触与自己无关的数据。因此,每个用户的外模式不一定相同。外模式是概念模式的一个子集或是从概念模式推导而来的。有了概念模式,设计外模式就比较方便。

3. 内模式: 内模式是用物理数据模型对数据的描述。概念模式与内模式之间可以进行映射,这个映射由 DBMS 来完成。内模式对用户是透明的,但是它的设计直接影响数据库的性能。因此,不但数据库的设计者和维护者应对内模式有充分的了解,数据库的用户最好对内模式也要有所了解,以便更好地使用数据库。

正如在三级数据模式的介绍中所述,概念模式描述数据库的逻辑,外模式是用户所看到的自己有权访问的数据部分的逻辑,内模式是用户数据在逻辑和物理上的存储,多级数据模式满足了不同用户和管理员对数据的需求,简化了访问和管理,提高了数据库的兼容性,使得数据的存储与使用独立,管理员和用户间数据模式独立,数据设计和使用独立。

2. 关系 R, S 如下图所示。试求下列关系代数运算结果(每小题 4 分,共 12 分)

R

1	2	3	4
a ₁	b ₁	c ₁	d ₁
a ₁	b ₁	c ₂	d ₂
a ₁	b ₁	c ₃	d ₃
a ₂	b ₂	c ₁	d ₁
a ₂	b ₂	c ₂	d ₂
a ₃	b ₃	c ₁	d ₁

S

1	2
c ₁	d ₁
c ₂	d ₂

(1) $\Pi_{3,4}(R) - S$

(2) $R \bowtie_c S, c = (R.3=S.1) \text{ AND } (R.4=S.2)$

(3) 用元组关系演算表示 $R \div S$

为了满足用户对数据的需求,简化访问和管理,避免数据冗余,修改时可能的不兼容,使得数据的存储与使用独立,管理员和用户的数据模式独立,修改和使用独立。

2. 关系运算分类结果..

σ (选择条件) (关系名): 6姓名 = '王师傅' (MASTER) —— 从括号中的关系选择满足条件的元组。

π (属性名) (关系名): π 姓名, 籍贯, 身份证号 (STUDENT) —— 从关系中选出这些属性的列即子集。
 $R \cup S$ 并操作, 取 R 和 S 的
 $R \cap S$

$R - S$ 差操作, 取 R 中不属于 S 的部分

$R \cap S$ 交操作, 取 R 和 S 中共同部分 $R \cap S = R - (R - S)$

$R \times S$ 笛卡尔积, R 和 S 中所有的拼接, 结果列为横数, 纵数为 $n_r + n_s$ 元组数为 $|R| \times |S|$ (R的元组数乘S的元组数)
 $R \bowtie S$ 满足特定条件的笛卡尔积。
(R的元组数乘S的元组数)

$R \div S$ 除法
 书上这里有个“1”

$$= \pi_x(R) - \pi_x(\pi_x(R) \times S - R)$$

2. (17) $\pi_{3,4}(R) \rightarrow S$

$\pi_{3,4}(R)$:

1	2
c_1	d_1
c_2	d_2
c_3	d_3

(要去掉重复的行)

$\pi_{3,4}(R) \rightarrow S$:

1	2
c_3	d_3

(2) $R \bowtie_c S, c = (R:3=S:1) \text{ AND } (R:4=S:2)$

R

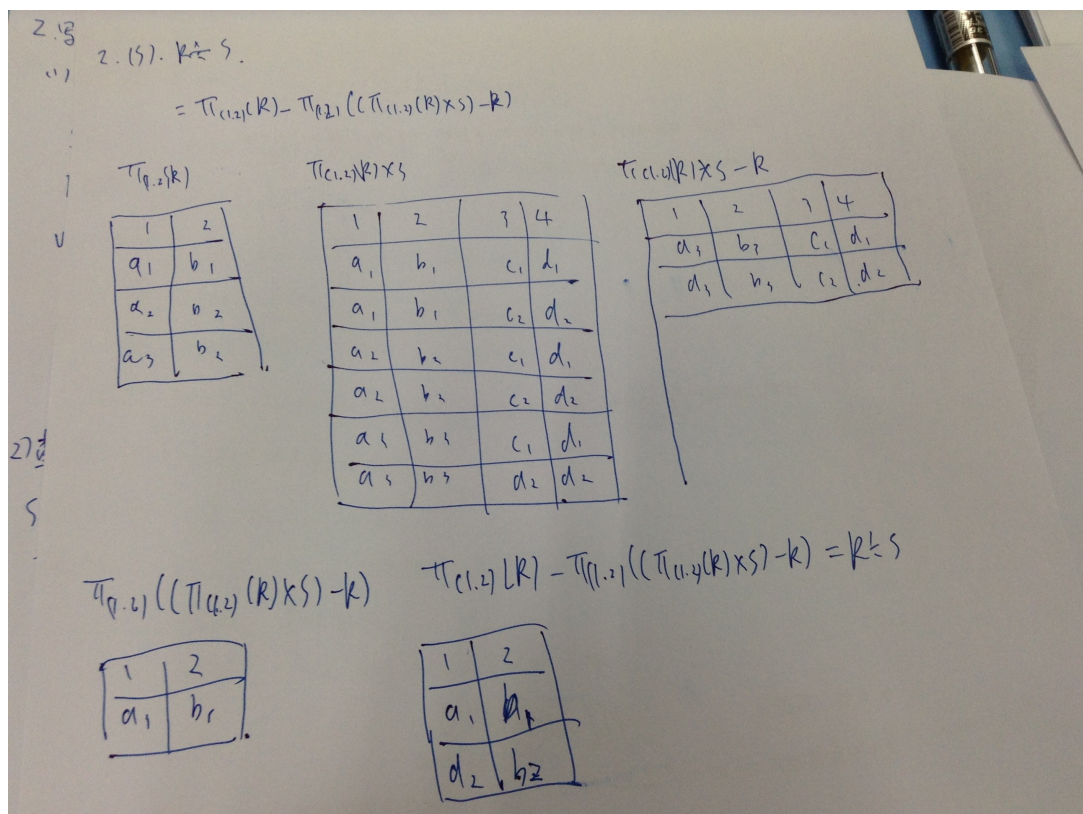
1	2	3	4
a_1	b_1	c_1	d_1
a_1	b_1	c_2	d_2
a_1	b_1	c_3	d_1
a_2	b_2	c_1	d_1
a_2	b_2	c_2	d_2
a_2	b_2	c_1	d_1

1	2
c_1	d_1
c_2	d_2

$R \bowtie_c S$:

1	2	3	4	5	6
a_1	b_1	c_1	d_1	c_1	d_1
a_1	b_1	c_2	d_2	c_2	d_2
a_2	b_2	c_1	d_1	c_1	d_1
a_2	b_2	c_2	d_2	c_2	d_2
a_2	b_2	c_1	d_1	c_1	d_1

print



2. 假设有下列三个关系(30%) :

Sailors(sid, sname, rating, birth, master) /*分别为水手的编号、名字、级别、出生日期、师父的编号，每个水手的师父也是水手*/

Boats(bid, bname, color)

/*分别为船的编号、名字、颜色*/

Reserves(sid, bid, day)

/*分别为订船水手编号、所订船编号、日期*/

试写出表达下列查询要求的 SQL 语句(必须用单条 SQL 语句表达) :

- (1) 用连接查询查预订了编号大于 103 的红色船的水手姓名 ;
- (2) 查询预订了所有红船的水手的编号 ;
- (3) 查询预订过的船只最多的水手的姓名 ;
- (4) 查询没有人预订的红船的名字 ;
- (5) 查询预订了 205 号船并且只预订过一次的水手姓名 ;
- (6) 按水手级别查询各级别水手预订红船的最大数目。

Select sname from sailors

Where sid in(select red-sailor.sid

From(select sailors.sname,sailors.sid,sailors.rating count(*) as num from reserves,sailors,boats

Where sailors.sid=reserves.sid and reserves.bid=boats.bid and boats.color='红'

Group by rating, sid) as red-sailor

Where

2. 写出 SQL 语句.

(1) 用连接查询预定编号大于 103 的红色船的水手姓名.

```
SELECT From Sailor sname AS 'Sailor Name'
FROM Sailors, Boats, Reserves.
```

```
WHERE Sailors.sidbid = Reserves.bid sid.
AND Reserves.bid = Boats.bid.
```

```
AND Boats.color = 'red'
```

```
AND Boats.bid > 103;
```

↓

```
AND Boats.bid > 103;
```

(2) 查询预订所有红色船的水手编号.

```
SELECT sid AS 'Sailor ID'
```

```
FROM (SELECT sid, COUNT(DISTINCT bid) AS cnt
FROM Reserves
```

```
WHERE bid IN (SELECT bid. from
```

```
FROM Boats
```

```
WHERE color = 'red')
```

```
GROUP BY sid) AS TEMP
```

```
WHERE cnt IN (SELECT count(sid)
```

```
FROM Reserves
```

```
WHERE color = 'red')
```


4.7

13) 查询预订过的船只最多水的姓名:

```
SELECT sname AS 'Sailor Name'
```

```
FROM Sails.
```

```
WHERE sid IN (SELECT MAX(cnt) sid
```

```
FROM (SELECT sid, MAX(cnt)
```

```
FROM (SELECT sid, COUNT(hid) AS cnt
```

```
FROM Reserves
```

```
GROUP BY sid)))
```

(4) 查询没有人预订的船只的姓名

```
SELECT bname AS 'Boat Name'
```

```
FROM Boats
```

```
WHERE bid NOT IN (SELECT bid  
FROM Reserves)
```

(5) 查询预订了205号船只且只预订一次的姓名。

```
SELECT sname AS 'Sailor Name'
```

```
FROM Sails.
```

```
WHERE sid IN (SELECT sid
```

```
FROM (SELECT sid, bid, COUNT(asid) AS cnt
```

```
FROM sails Reserves
```

```
WHERE bid=205
```

```
GROUP BY sid)
```

```
WHERE cnt=1)
```


16. 按水手级别查询各级别水手预订红船的数量。

```

SELECT rating AS 'Sailor's Rating', MAX(cnt) MAX cnt AS 'Number of Boats'
FROM Sailors, (SELECT sid, COUNT(sid) AS cnt
                FROM Boats.Reserves
                WHERE Boats.bid = Reserves.bid
                AND color = 'red'
                GROUP BY sid) AS Num
GROUP BY rating;
  
```

2. 利用上题中的关系，试用嵌有 SQL 的 C 语言程序打印一张报表，内容是级别、该级别水手的平均年龄 $\bar{b}$ ，假设 Sailors 表中 birth 属性类型为日期型。（只需表明访问数据库及对查询结果进行处理的程序逻辑，不需要严格编程）(10%)。

```

}
// 查询结果语句, 定义游标
EXEC SQL DECLARE C1 CURSOR FOR
    SELECT rating, AVG(YEAR(birth) YEAR(birth))
    FROM Sailors
    GROUP BY rating;

EXEC SQL OPEN C1;
printf
write(TRUE)
printf("Sailor's Rating    Average of Age\n");
int rate, age;

while(TRUE)
{
    EXEC SQL FETCH C1 INTO :rate, :age;
    if (SQLCA.SQLCODE = 100 || SQLCA.SQLCODE)
        break;
    printf(rate);
    printf(" ");
    printf(age);
    printf("\n");
}
EXEC SQL CLOSE C1;

```

4. 试为第2题中的 Reserves 关系定义一个完整性约束条件：不允许预订绿船(5%)。

4. 不允许预订绿船。
触发子实现。

```

GR CREATE TRIGGER reserve_greenboat_never
BEFORE INSERT ON Reserve
REFERENCING NEW AS N
FOR EACH ROW
WHEN C SELECT EXISTS (SELECT bid *
    FROM N, Boats
    WHERE N.bid = Boats.bid
    AND Boats.color = 'green')
ROLLBACK;

```

用此实现 ASSERT reserve_greenboat_never ON Reserves: bid NOT IN (SELECT bid FROM Boats WHERE ~~Boats~~ color = 'green');

5. (索引/查询优化)?

(索引/查询优化) 对索引记录上一检查与以前的已提交事务应该记录否?

5. (索引/查询优化) (10%)?

6. 介质失效恢复时, 对运行记录中上一检查点以前的已提交事务应该 redo 否? (5%)

7. 举例说明什么是分布式数据库系统并发控制中的全局死锁。(10%)?

用 SQL 实现 ASSET reserve-granboot-move ON Reserver: bid NOT INC

5. (索引/查询优化)?

```
SELECT bid
FROM Boats
WHERE Boats
(color='green');
```

附 6. 介质失效恢复时, 对运行记录中上一检查点以前的已提交事务应该 redo 否?

介质失效指磁盘发生故障, 数据库受损如划盘、磁头故障等。

首先, 介质失效后应在新介质中加载最近的备份副本。假设检查点比备份副本新, 则对于备份副本以前的已提交的事务不应 redo。对备份副本以后, 检查点以前的提交的事务应该 redo。则此系统可恢复至检查点这一一致状态。

7. 举例说明什么是分布式数据库系统并发控制中的全局死锁。

死锁是在并发控制时需要和掌握资源的事务之间循环等待。

举例: 客户端 A 向服务器提交事务 Ta。Ta 需要 R₁, R₂, R₃, R₄, R₅ 5 个资源, 并已申请到 R₁ 和 R₂。

客户端 B 向服务器提交事务 Tb。Tb 需要 R₃, R₄, R₅, R₆, R₇ 5 个资源, 并已申请到 R₅ 和 R₆。

至此, Ta 和 Tb 的循环等待开始, 全局死锁产生。

8. 编写一个触发器, 监视第 2 题 Sailors 表上的 Update 操作, 对每条 Update 语句, 判断其更新后的元组是否有 1990.1.1 之后出生的水手, 将这样的水手自动插入到 YoungSailors 表中 (YoungSailors 表与 Sailors 表的模式相同)。(10%)

6. 编写一个触发器

CREATE TRIGGER check-young-sailor
AFTER UPDATE ON Sailors.

~~FOR EACH STATEMENT ROW.~~

REFERENCING NEW AS N.

FOR EACH ROW.

WHEN (EXISTS (SELECT *
FROM Sailors.

WHERE YEAR(birth) > 1990.)

INSERT INTO YoungSailors

SELECT s.id, s.name, rating, birth, master
FROM Sailors.

WHERE YEAR(birth) > 1990;

附加题：试分析分布式数据库系统出现的技术背景和应用背景。它与后来出现的联邦式数据库系统的类似之处和本质区别是什么(10%)？


```
SELECT k.id, s.name, rating, birth, master
FROM Sailors.
WHERE YEAR(birth) > 1940;
```

附加题：分布式数据库出现的背景是它后来出现的联邦式数据库系统的类似之处和本区别是什么？

因为集中式数据库有以下缺点：通信开销大，可扩充性差，性能瓶颈，难以管理，可用性差。

所以后来希望产生一种物理上分布，逻辑上集中的分布式数据库，即把全局数据模式按数据的来源和用途合理分布到各个物理节点上，逻辑上，用户看到的是一个数据模式为全局数据模式的集中式数据库。

“联邦式数据库系统”是分布式数据库系统的一种。

相似之处为物理上分布，全局数据模式分布在不同的节点上。

不同之处为：1) 节点自治，每个节点有本节点的数据模式。

2) 没有全局模式。

3) 每个节点上有供本节点共享的其它节点上有关的数据模式。

4) 节点数据模式的修改甚至节点的加入、删除、对量均有影响节点。